Multi-mode SDR Decoder: Learning Roadmap

Nov 3, 2026 · @Zakary Chauve

Project goal and scope

The goal is one Python program that tunes an RTL-SDR to any frequency, lets you pick a mode, and turns the signal into sound or decoded data. You build it in seven phases, each ending with something that works on its own.

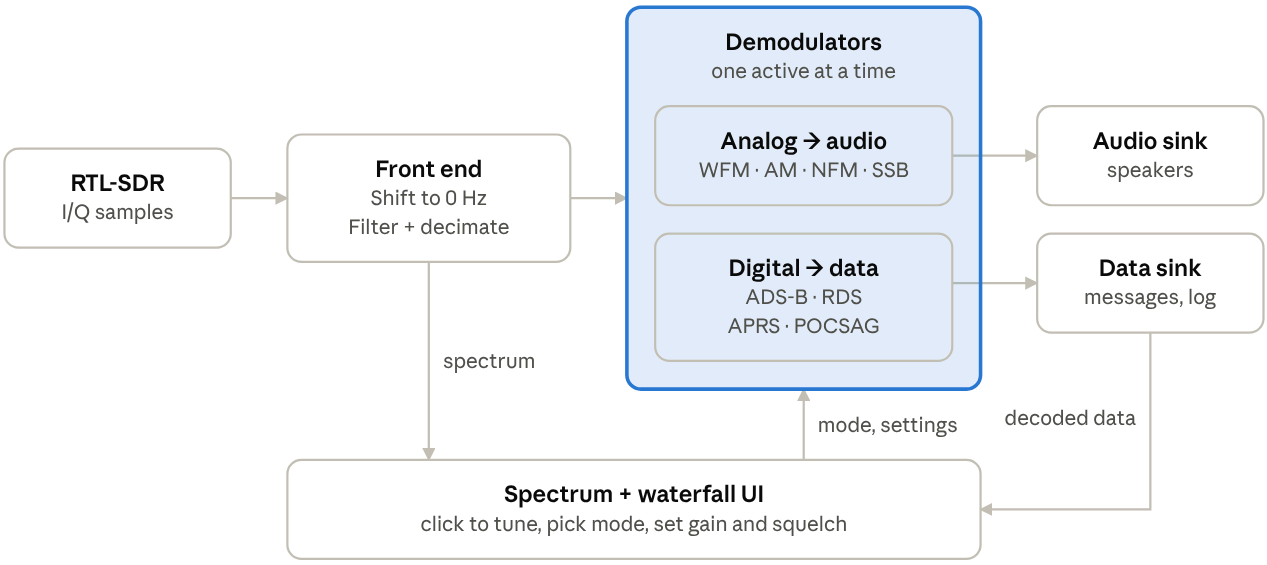
Modes covered, in the order you'll build them:

- **Analog voice:** wideband FM (broadcast radio), AM (air band), narrowband FM (walkie-talkies, ham), SSB (ham shortwave)
- **Digital data:** RDS (station names on FM radio), ADS-B (aircraft positions), APRS (ham packet radio), POCSAG (pagers)

The core idea: every mode shares the same front end (read samples, shift, filter, decimate). Only the last stage, the demodulator or decoder, changes. If you design around that from Phase 5, adding a mode becomes adding one small module rather than rewriting the program.

The architecture diagram at the end of this section shows how the parts fit together.

Every mode shares one front end; only the demodulator changes



decoder architecture · shared front end, swappable demodulators

Samples flow left to right; the UI taps the spectrum and sends mode and settings back to whichever demodulator is active.

Prerequisites

You already have the hardware and a working spectrum display, so you're past step zero. Make sure these are solid before Phase 1:

- **Python:** functions, classes, modules, and reading other people's code
- **numpy:** array slicing, vectorized math (no per-sample `for` loops), `np.fft`
- **Complex numbers:** what magnitude and phase (angle) mean, and that multiplying by `e^(jθ)` rotates a point. This is the single most important bit of math in SDR.
- **Basic trigonometry:** sine waves, frequency, phase
- **Tools:** pyrtlsdr, numpy, scipy, matplotlib, sounddevice installed and working

If complex numbers feel shaky, spend an evening on them first. Every later phase depends on reading I/Q samples as rotating points.

Phase 1: DSP fundamentals

Learn the five ideas every decoder is built from. Test each one on synthetic signals you generate in numpy, before touching the radio.

Learn:

1. **Sampling and the Nyquist limit:** why 2.4 MHz of complex samples covers a 2.4 MHz wide band.
2. **I/Q samples:** why the dongle gives complex numbers, and how positive and negative frequencies sit either side of the center.
3. **The FFT:** turning samples into a spectrum, bins, resolution, windowing. You already use this through `psd()`.
4. **Frequency shifting:** multiplying by a complex tone to move a signal to 0 Hz.
5. **Filters, decimation and aliasing:** FIR low-pass filters (`scipy.signal.firwin`, `lfilter`), why you must filter before throwing samples away.

Build:

- [ ] Generate a sine wave, plot its FFT, and find its frequency
- [ ] Generate two tones, shift one to 0 Hz, filter out the other
- [ ] Decimate with and without a filter and see the aliasing yourself

Done when: you can explain, without notes, why the order is shift, filter, then decimate.

Phase 2: First decoder, offline FM radio

Decode a recording of an FM station into a WAV file. Working offline removes all timing problems, so you can focus on getting the signal chain right.

Learn:

- How FM encodes audio in the rate of phase change
- The phase-difference demodulator: the angle between each sample and the previous one
- De-emphasis (50 µs in Europe) and why skipping it sounds harsh
- Resampling to 48 kHz for audio

Build:

- [ ] Record about 10 seconds of raw I/Q around a strong local station (88–108 MHz) and save it with `np.save`

- [ ] Tune 250 kHz off the station to avoid the DC spike, then shift it back digitally
- [ ] Filter to ±100 kHz, decimate 2.4 MHz → 240 kHz
- [ ] Demodulate, de-emphasize, decimate to 48 kHz, normalize
- [ ] Write a WAV file with `scipy.io.wavfile.write` and listen

Done when: the WAV plays clean music or speech. Keep the recording; it's your test file for everything after.

Debugging tip: plot the spectrum after every stage. If the audio is noise, the plots show you which stage broke it.

Phase 3: More analog modes

Add AM, narrowband FM and SSB, still offline. You'll notice that 80% of your Phase 2 code is reused, which is the insight Phase 5 is built on.

Mode	Where to find it	Bandwidth	Demodulation
AM	Air band, 118–137 MHz	about 8 kHz	Magnitude of each sample, then remove the DC offset
NFM	PMR446 (446 MHz), ham 2 m (144–146 MHz)	about 12.5 kHz	Same as wideband FM, narrower filter, no stereo or broadcast de-emphasis
SSB (USB/LSB)	Ham bands; HF needs an upconverter or direct-sampling mode	about 3 kHz	Shift, keep only one side of the spectrum, take the real part

Learn:

- Envelope detection for AM, and an AGC (automatic gain control) to even out the volume
- Squelch: muting the audio when the signal power drops below a threshold, essential for NFM and air band where transmissions are short
- Single sideband and why it sounds like a duck when tuned slightly wrong

Build:

- [ ] AM demodulator + AGC, tested on an air band recording
- [ ] NFM demodulator + squelch

- [ ] SSB demodulator with a fine-tuning offset

Done when: each mode produces understandable audio from a recording. Air band is quiet most of the time, so record near an airport or during busy hours.

Phase 4: Real-time streaming

Move from "process a file" to "listen live." This is the hardest engineering step of the project, not because of the math but because of timing.

Learn:

- **Block processing:** the radio delivers chunks; your pipeline handles one chunk at a time
- **Filter state:** `scipy.signal.lfilter` returns its internal state (`zi`); pass it into the next block or you hear a click at every boundary. The FM demodulator also needs the last sample of the previous block.
- **Producer/consumer threads:** one thread reads the radio, one processes, the audio callback plays, connected by `queue.Queue`
- **Buffers and latency:** too small and the audio stutters, too large and tuning feels laggy
- **Performance:** each block must be processed faster than it arrives. Profile with `time.perf_counter()`.

Build:

- [ ] Read samples asynchronously with `sdr.read_samples_async` or a reader thread
- [ ] Rewrite your FM chain to keep filter state between blocks
- [ ] Play through a `sounddevice.OutputStream` callback fed from a queue
- [ ] Log queue length; it should stay roughly constant

Done when: you can listen to FM radio live for 10 minutes without clicks or dropouts.

Tip: verify your block-based chain against Phase 2 first. Run the recording through it in blocks; the WAV should be identical to the one-shot version.

Phase 5: Modular multi-mode architecture

Restructure the code so every mode plugs into the same pipeline. After this phase, a new mode is one new file.

Learn:

- Classes and a shared interface (Python's `abc.ABC` or a simple base class)
- Separating configuration from code (a dict or a JSON/YAML file of mode presets)
- Swapping a component safely while threads are running (a lock, or a mode-change message on the queue)

Design:

- A `FrontEnd` that owns the radio, the frequency shift, and the first filter + decimation
- A `Demodulator` base class with one method, `process(iq_block) -> output`, plus its own `sample_rate`, `bandwidth` and state
- One subclass per mode: `WFM`, `AM`, `NFM`, `SSB`; later `RDS`, `ADSB`…
- Two kinds of output: an `AudioSink` (sounddevice) and a `DataSink` (print, log file, or the UI)
- A preset table per mode: default bandwidth, decimation, de-emphasis, squelch

Build:

- [ ] Refactor Phases 2–4 into this structure without changing behavior
- [ ] Switch mode and frequency from the keyboard while running
- [ ] Write a unit test per demodulator that runs your saved recordings

Done when: switching between WFM, AM, NFM and SSB works live, and adding a mode touches no existing file.

Phase 6: User interface

Give the decoder a window: a live spectrum, a waterfall, and controls. Your current matplotlib script is the starting point.

Learn:

- **Waterfall displays:** a 2D image where each new row is the latest spectrum, scrolling over time
- **A faster plotting library:** matplotlib is fine for a few updates per second; for a smooth waterfall use **pyqtgraph** (built on Qt, designed for real-time data)
- **GUI basics:** widgets, signals and slots, keeping the GUI thread separate from DSP threads

Build:

- [ ] Spectrum + waterfall view fed from the front end

- [ ] Click on the waterfall to tune to that frequency
- [ ] Mode dropdown, bandwidth, gain, volume and squelch controls
- [ ] A text panel for decoded data (used by Phase 7)
- [ ] Optional: a frequency bookmarks list

Done when: you can find a signal on the waterfall, click it, pick its mode, and hear it, without touching the code.

Phase 7: Digital modes

Digital decoders add three steps after demodulation: recover the symbol timing, turn symbols into bits, then parse the bits into messages and check them. Build them in this order, easiest first.

Mode	Frequency (Europe)	Modulation	What you get	Main new skill
ADS-B	1090 MHz	Pulse position, 1 Mbit/s, 112-bit messages	Aircraft ID, altitude, position	Detecting a preamble, CRC checks
RDS	Inside FM broadcast, 57 kHz subcarrier	BPSK, 1187.5 bit/s	Station name, radio text	Carrier and symbol recovery, block sync
APRS	144.800 MHz	AFSK 1200 baud on NFM	Ham station positions and messages	Tone detection, HDLC/AX.25 framing
POCSAG	Varies by country	2-FSK, 512/1200/2400 baud	Pager messages	FSK slicing, BCH error correction

Learn:

- Symbol timing recovery (start simple: oversample and pick the best phase; later Mueller-Müller or Gardner)
- Bit slicing, framing and sync words
- Error detection and correction: CRC, parity, BCH
- Reading protocol specs: each mode has a public specification, and decoding is mostly careful bit bookkeeping

Build:

- [ ] ADS-B: set the dongle to 2 MHz at 1090 MHz, detect preambles in the magnitude, decode the aircraft ID. Compare with a site like flightradar24 to check.
- [ ] RDS: extend your WFM decoder to extract the 57 kHz subcarrier and print the station name
- [ ] APRS: feed NFM audio into an AFSK demodulator and decode AX.25 frames
- [ ] Each one as a `Demodulator` subclass writing to the `DataSink`

Done when: at least two digital modes decode correctly. Validate against an existing decoder (dump1090, Direwolf, multimon-ng) on the same recording.

Learning resources and tools

Start with PySDR; it covers Phases 1–3 and parts of 7 in Python, with code close to what you're writing.

Resource	Type	Best for
PySDR	Free online textbook	I/Q, filters, FM, digital modulation, synchronization, all in Python
The Scientist and Engineer's Guide to DSP	Free online book	Clear explanations of FFT and filters
Understanding Digital Signal Processing (R. Lyons)	Book	The reference for DSP intuition
GNU Radio tutorials	Wiki	Seeing a full chain as blocks before coding it
rtl-sdr.com	Blog	Project ideas, hardware tips, what's receivable
SDR++	Ready-made SDR app	Finding signals and checking what yours should sound like

Reference decoders to compare against (read their source when stuck): dump1090 (ADS-B), Direwolf (APRS), multimon-ng (POCSAG and others), redsea (RDS).

Libraries: pyrtlsdr, numpy, scipy.signal, sounddevice, pyqtgraph.

Common pitfalls and a legal note

Most stuck moments in SDR projects come from a handful of causes. Check these first.

- **No signal at all:** standard RTL-SDR dongles cover roughly 24 MHz to 1.7 GHz; 2.4 GHz is out of range for most
- **Spike in the middle of the spectrum:** the DC spike; tune off-center and shift digitally
- **Garbled or doubled audio:** decimating before filtering (aliasing)
- **Clicks every fraction of a second:** filter state not carried between blocks
- **Stuttering live audio:** processing slower than real time; vectorize, decimate earlier, or use bigger blocks
- **Frequency slightly off:** cheap dongles drift; set `sdr.freq_correction` (ppm), or buy a TCXO model
- **Gain too high:** strong signals overload the receiver and create fake signals everywhere; lower the gain
- **"Device busy" on restart:** the previous run didn't call `sdr.close()`; keep the `try/finally`

Legal note: receiving broadcast radio, air band, ADS-B and amateur radio is generally fine. Rules on intercepting other communications, and especially on recording or sharing what you hear (pagers, private radio), vary by country. In Switzerland, check OFCOM (BAKOM)'s rules before decoding anything beyond public broadcasts and ham radio.